PROJECT HANDBOOK | ARCHITECTURE | WORKFLOW

SGNIA Project Guide

System, features, technology and workflow

Sindh Government Notification Issuing Authority — university prototype

This guide describes repository behavior and distinguishes it from official policy.
It is not an approved Government procedure or service.
The software model is not a legal delegation of authority.

At a glance

SGNIA is a university-project prototype combining a public-facing archive with a password-protected staff workspace. The public side indexes Government of Sindh source listings and project-published records. The staff side supports drafting, review, publication, document versioning, a project signature, OCR assistance and an audit trail. The application is not an official Government service.

The application models a review workflow; it does not encode or prove the Government of Sindh legal approval chain. Actual signatory powers, delegation rules and document procedures must come from the applicable official rules.

How to use this guide

- Sections 1–4 explain architecture, languages, libraries and public/staff features.
- Sections 5–7 explain software roles, status flow, departments and the government-process boundary.
- Sections 8–12 explain security meaning, deployment readiness, review findings and repository locations.

1. System architecture

The repository has two connected experiences. Static web assets provide the archive interface. Flask serves those pages, the staff workspace and an API for published staff records. The browser also reads published rows and departments from Supabase REST using the public anon role. The archive merges Supabase and Flask responses; Flask remains the staff workflow publication path.

Layer	What it does	Implementation
Visitor	Homepage, archive, QR lookup and local PDF hash comparison	HTML, CSS, vanilla JavaScript
Staff	Login → draft/version → review → approve → publish	Flask, Jinja templates, server-side permissions
Data	Flask relational models plus Supabase REST read integration	PostgreSQL/Supabase; SQLite local; MySQL adapter
Runtime	Schema preparation and Gunicorn start in a container	Docker, Python, Gunicorn, Render blueprint

Data-source distinction: Flask ORM models use singular entities such as user, notification and notification_version. Browser Supabase REST reads the plural public.notifications and public.departments tables. Merging responses in the browser does not replicate data or create one source of truth. Confirm which store is authoritative before operational use.

Repository map

Location	Purpose
index.html, archive.html, styles.css, theme.js	Public pages, visual styling, theme preference and accessible structure.
app.js, archive.js, records.js, public-records.js	Search, filters, QR scanning, PDF comparison, bundled and live records.
supabase-config.js, supabase_schema.sql	Supabase public REST client, schema, RLS and seed data.
staff_portal/app/staff.py, public.py, auth.py	Staff routes/workflow, public API and login.
models.py, permissions.py, signatures.py, audit.py	Entities, role matrix, project signatures and audit chain.
staff_portal/app/templates/	Jinja staff pages, forms, archive and signature status.
staff_portal/migrations/, tests/	Schema migrations and Python tests.
Dockerfile, render.yaml, requirements.txt	Container, deployment blueprint and runtime dependencies.

2. Technology stack and languages

Language / format	Where and why it is used
Python	Flask backend, forms, access checks, OCR integration, PDF creation, signatures, migrations and tests.
HTML + Jinja2	Public page structure and server-rendered staff forms, tables, errors and role-aware navigation.
CSS	Responsive layouts, themes, spacing, typography and mobile presentation.
JavaScript (vanilla)	Search, filters, pagination, live record loading, QR scanning, theme and browser-local file hashing.
SQL	PostgreSQL/Supabase schema, constraints, indexes, RLS policies and migrations; ORM also supports SQLite/MySQL.
YAML	Render service blueprint and deployment environment variable declarations.
Docker / shell	Container build, OCR system packages, schema preparation and Gunicorn startup.

Library / runtime	Purpose
Flask, Flask-Login, Flask-WTF, WTForms	HTTP routes, login sessions, CSRF protection and validated forms.
SQLAlchemy, Flask-Migrate / Alembic	Relational models and tracked schema changes.
PostgreSQL driver, PyMySQL, SQLite	Hosted Postgres/Supabase and supported local/alternate database connections.
PyMuPDF, pypdf, Tesseract, pytesseract	PDF generation/reading, text extraction and OCR for images or scans.
cryptography (Ed25519), hashlib	Project signatures and SHA-256 integrity fingerprints.
qrcode, Pillow	Project QR images and image processing.
Gunicorn, Docker, Render	WSGI server, container packaging and deployment blueprint.

Runtime note: the Docker image uses Python 3.12, while repository/local setup documentation pins Python 3.11 and also states Python 3.10+ as the minimum. Keep runtime declarations aligned and test upgrades against the suite.

3. Public portal features

Feature	What it does / benefit
Searchable archive	Search title, reference or portal listing; filter by department/date; sort and paginate.
Source-linked records	Bundled listings link to source pages/PDFs from the Government of Sindh website.
SGNIA project records	Published staff-workflow records appear in the public archive after publication.
PDF comparison	Hashes a chosen PDF in the browser and compares it with a stored fingerprint; this check does not upload the file.
QR scanning	Uses native BarcodeDetector or jsQR fallback; QR payloads are matched without auto-opening arbitrary links.
Project QR download	QR codes open a project archive lookup; they are not Government-issued credentials.
Responsive/accessibility	Keyboard-aware controls, skip links, labels, live result regions and device-sized layouts.
Themes	Shared light/dark selection with browser preference persistence.

A hash match means the selected bytes equal the copy currently stored for that project record. It does not prove that a Government authority created, signed or issued it.

4. Staff workspace features

- Authenticated login with password hashes, protected server routes, role checks and CSRF-protected forms.
- Create, edit, submit and review department-scoped drafts; optionally upload a PDF or generate a marked project demonstration PDF.
- Local PDF/image OCR; staff review and confirm extracted text before saving it as a draft.
- Approval, return, rejection and a distinct publication step; only Published records are exposed by the Flask public API.
- Numbered amendments; a replacement is reviewed and published before it supersedes the current public version.
- Project Ed25519 keys sign the record reference, version and PDF fingerprint. Private keys are filesystem-based.
- Hash-chained audit events; administrators can review the permission matrix and chain status.
- Staff account management, department assignment, password reset, active/inactive state and key management.

Benefit: a traceable draft-to-publication interface, private pending work, version history and useful file-integrity checks. This is not a records-retention certification or a Government document-management system.

5. Roles and permissions

Software role	Implemented capability (summary)
Super Admin	Full admin matrix; designated protected account for system administration and key operations.
Admin	Manage staff/roles/departments and perform allowed drafting, review, publication, audit and key tasks.
Approver	Can create drafts, review/approve/return/reject, publish, use OCR, and view archive/history/audit.
Creator	Can create/submit drafts and use OCR; scoped to assigned department/own documents.
Auditor	Read-focused record/archive and audit access; no create, approve or publish permission.

These are application roles, not Government job titles. “Approver” does not automatically mean Secretary, Minister, competent authority or legally delegated signatory.

Permission overview

Permission	Super Admin	Admin	Approver	Creator	Auditor
Create/draft	Yes	Yes	Yes	Yes	No
Approve/return/reject	Yes	Yes	Yes	No	No
Publish	Yes	Yes	Yes	No	No
Manage staff/roles	Yes	Yes	No	No	No
Manage signing keys	Yes	Yes	No	No	No
View audit trail	Yes	Yes	Yes	No	Yes

Approver also has create and publish permission. The current application does not enforce that the reviewer differs from the creator. If separation of duties is required, define the rule and enforce it explicitly.

6. Application workflow: create to publish

Stage	Who (software role)	What happens
1. Draft	Creator, Approver, Admin or Super Admin	Enter title, type, department, subject, date and body; upload PDF or generate a marked demo PDF.

Stage	Who (software role)	What happens
2. Submit	Draft owner / authorized administrator	Draft or Returned moves to Pending Approval; pending data remains private.
3. Review	Approver, Admin or Super Admin	Approve → Approved; Return → Returned; Reject → Rejected. Approval alone does not publish.
4. Publish	Role with publish permission	Requires versioned PDF, active project signing key and matching stored PDF hash.
5. Public record	Public visitor	Published rows appear in API/archive; project PDF, QR lookup and signature status are available.
6. Amendment	Authorized staff	Creates a numbered version; existing publication remains active while amendment is reviewed.

Status path: Draft → Pending Approval → Approved → Published. Returned work can be submitted again; rejected records are not published. Amendments use the same stages, and the previous public version remains current until its replacement publishes.

Government process versus software workflow

A generic administrative process might include an originating section, departmental review, legal/finance consultation when applicable, approval by an authorized competent authority, signature, issue/dispatch and record retention. The correct sequence depends on law, delegation, document type, department rules and subject. This project does not implement these offices, consultative stages, dispatch registers or statutory approval rules. The diagram is illustrative only.

Illustrative only; not a prescribed Government of Sindh chain.

7. Departments, sub-departments and document types

The Department model has a self-referencing parent_id, so a parent/child hierarchy can be stored. User assignment and notification routing primarily use department-name strings. There is no complete sub-department assignment system, inherited permissions model, section/wing routing tree or multi-stage sub-department approval chain.

Letter, Notification, Circular, Advisory and Order are selectable categories in a shared form and common status workflow. The project does not attach a distinct legal authority matrix, mandatory consultation sequence, template-signatory rule or special publication policy to each category.

The public frontend and Supabase schema include separate department datasets. README references a 29-department filter list, while the SQL seed has its own list. Confirm the authoritative current data with the project owner; do not treat seed descriptions as official policy.

8. Security, privacy and what “verified” means

Control / feature	Meaning and limits
Authentication	Passwords are hashed; staff sessions use Flask-Login and secure session settings in production.
Authorization	Role, department and author scope are checked on server routes; menu visibility is not the security boundary.
CSRF and uploads	Flask-WTF CSRF protection; configured request limits and PDF signature checks.
CNIC privacy	Keyed digest and masked display; protect lookup key and keep it stable or migrate deliberately.
Database RLS	Supabase schema enables RLS and public SELECT policies on selected tables; review live grants, policies and views.

Control / feature	Meaning and limits
SHA-256	Detects whether file bytes match a stored fingerprint; not authorship or Government issuance.
Ed25519	Project key signs reference/version/hash; not Government PKI or proof of Government authority.
Audit chain	Detects many edits, deletion and reordering; a privileged DB operator can rebuild the entire chain.
QR code	Project-generated lookup link; not encrypted proof, Government-issued identifier or legal authenticity check.
OCR	Local Tesseract processing; extracted text requires human review and can contain recognition errors.

Never put a Supabase service-role key, Flask secret, database password or private signing key in browser code or a tracked manifest. The anon key is public by design only when RLS and grants are correctly configured.

9. Local setup and production readiness

Local developer setup

- Follow staff_portal/README.md PowerShell instructions to create a virtual environment and install requirements.
- Copy .env.example to .env, set a private local FLASK_SECRET_KEY, then run prepare-db, seed-departments and create-user.
- Start with python -m flask --app run.py run from staff_portal; localhost:5000 serves the public site and /login serves staff.
- Install Tesseract locally to exercise OCR for images or scanned PDFs; normal drafting does not require OCR.

Production checklist

Area	Required action before real use
Secrets	Set host-managed FLASK_SECRET_KEY and DATABASE_URL. Rotate any previously exposed secret; removal from the current file does not erase Git history.
Admin	Set SUPER_ADMIN_EMAIL and verify the intended protected account before migrations.
Storage	Use durable, access-controlled NOTIFICATION_UPLOAD_DIR and SIGNING_KEY_DIR; restrict private-key access and back up securely.
Render	Blueprint selects production and marks settings manual. No persistent disk is configured and the service plan is free; do not use ephemeral storage for documents/keys.
Database	Back up before schema migration; ensure connection mode supports startup migrations; test least privilege and live RLS/view behavior.
Operations	Configure HTTPS, test secure cookies and restore from encrypted backups, verify signatures after restart, and run monitoring/tests.

No deployment or live database verification was performed in this review. The Render manifest change is source configuration only; dashboard variables, storage, key generation, credentials and deployment must be completed separately.

10. Review findings and current status

Status	Finding / next step
Fixed in source	Approval no longer jumps from Pending Approval to Published; a separate Approved → Published action is used.
Fixed in source	Publishing blocks if the versioned PDF or project signing requirements fail; user-facing signing errors are generic.
Fixed in source	The archive merges Flask API records even when Supabase returns an empty list.

Status	Finding / next step
Fixed in source	Generated PDFs and PDF comparison now state that project output is not Government-issued and matching hashes are not proof of authenticity.
Fixed in source	Staff docs match optional upload behavior; the account-creation test supplies the required CNIC.
Configured, not deployed	Render selects production; database, app secret, admin email and persistent paths need manual host configuration.
Owner decision	Decide whether a user may approve their own draft; no independent-reviewer rule is enforced.
Owner decision	Choose authoritative public data, current department list, sub-department routing and document-specific approval matrix.
Operational action	Provision durable storage, rotate old secret, back up database/files/keys and test recovery before deployment.
Review boundary	Live Supabase grants/RLS/views, hosting dashboard values, source accuracy and official procedures were not independently verified.

Validation completed: 48 staff portal tests passed. JavaScript syntax, public-record merge behavior and Render YAML were also checked.

11. Benefits and limitations

Benefits

- Public search makes source-linked records easier to locate without a staff login.
- A role-aware workspace organizes drafting, review, publication and amendment history.
- Department scope and server-side authorization reduce accidental cross-department access.
- Versioning, project signatures, hashes and chained audit events add prototype-level traceability.
- OCR and browser hashing are local, without a third-party OCR or hash service.

Limitations

- No Government authorization, official records status, legal delegation or service-level commitment is established.
- No full sub-department routing or document-type-specific legal approval logic.
- No enforced separation between creator and approver; admin/approver capabilities are broad.
- Public read data can come from more than one source; reconciliation and ownership need explicit design.
- Persistent storage, backups, restore drills, monitoring and live-production configuration remain necessary.
- A project signature, QR lookup or matching file hash cannot be presented as Government authenticity.

12. Key terminology and source files

Term	Project meaning
Archive reference	Project search identifier; not automatically an official Government reference.
Creator / Approver	Application permissions; names do not prove legal authority.
RLS	PostgreSQL Row-Level Security, which filters table access according to role and policy.
Security invoker view	PostgreSQL view mode that evaluates underlying permissions/RLS as the querying role, when supported and granted.
SHA-256	Data digest for byte comparison; not a signature or identity proof.
Ed25519	Public-key signature algorithm used with project-managed keys, not Government PKI.

Repository references

- README.md and staff_portal/README.md
- staff_portal/app/staff.py, public.py, models.py, permissions.py, signatures.py and pdf_generator.py
- index.html, archive.html, app.js, archive.js, public-records.js, records.js and supabase-config.js
- supabase_schema.sql, staff_portal/migrations/, tests/, Dockerfile and render.yaml

This is a technical explanation of repository behavior, not legal advice, policy approval, cybersecurity certification or Government authorization.

Confirm operational decisions, source data, permissions, Government process requirements and hosting controls with the responsible project owner before production use.